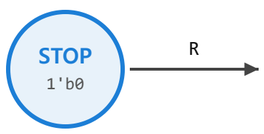
컨트롤 유닛 FSM — 상태도 · ASM 차트

verification_v2 · Top (stopwatch_watch) 안의 컨트롤 유닛 3 개 · 원본 source_file/*_control_unit.sv

쪽	인스턴스 (파일)	상태 = 코드	입력	출력
2 · 3	U_STOPWATCH_CONTROL_UNIT (stopwatch_control_unit.sv)	STOP = 1'b0 (리셋) RUN = 1'b1	i_sw_btn_R i_sw_btn_L i_sw_btn_U	o_sw_run_stop o_sw_clear o_sw_down
4 · 5	U_TIMER_CONTROL_UNIT (timer_control_unit.sv)	SET = 2'b00 (리셋) RUN = 2'b01 ALARM = 2'b10	i_tm_btn_R i_tm_btn_L i_tm_btn_U i_tm_btn_D i_tm_zero	o_tm_run o_tm_alarm o_tm_clear o_tm_add o_tm_amount
6 · 7	U_WATCH_CONTROL_UNIT (watch_control_unit.sv)	NORMAL = 2'b00 (리셋) SET_HOUR = 2'b01 SET_MIN_10 = 2'b10 SET_MIN_1 = 2'b11	i_wt_btn_R i_wt_btn_L i_wt_btn_U i_wt_btn_D i_set_sw2	o_wt_cursor o_wt_up o_wt_down

읽는 법



상태도: 원 = 상태 (이름 · enum 코드 · Moore 출력)
화살표 = 그 조건이 1 이면 다음 클럭 엣지에 옮겨 감 (조건은 약어, 쪽마다 약어표)
오른쪽 표 = 같은 내용을 포트 이름 그대로 (다음 상태 표는 위 줄이 우선)

 상태 박스: 상태 이름 · 코드 + Moore 출력 (그 상태에 있는 동안 계속)

 판단: 입력 조건 (1 / 0 갈래)

 조건 출력 (Mealy): 그 클럭에 바로, 1 클럭 동안

 다음 상태 (다음 클럭 엣지에 c_state 가 됨)

ASM 차트: 상태 1 개 = ASM 블록 1 개 = 한 클럭. 블록 안의 판단 순서 = RTL 의 if / else if 순서.

타이밍 (Vivado 파형에서 c_state · n_state 로 보기)

- n_state 는 조합 회로: 버튼 펄스 (10 ns) 가 들어온 그 클럭 동안 바로 바뀐다
- c_state 는 레지스터: 다음 클럭 엣지에 n_state 값이 되고, 이때 Moore 출력 (o_sw_run_stop, o_tm_run, o_wt_cursor …) 이 바뀐다
- Mealy 출력 (o_sw_clear, o_tm_add, o_tm_clear, o_wt_up, o_wt_down) 은 입력이 1 인 그 클럭에만 1 → 1 클럭 펄스
- 버튼 입력은 debounce 뒤 1 클럭 펄스라 같은 클럭에 둘이 겹치는 일은 드물지만, 겹쳐도 차트의 우선순위대로 동작

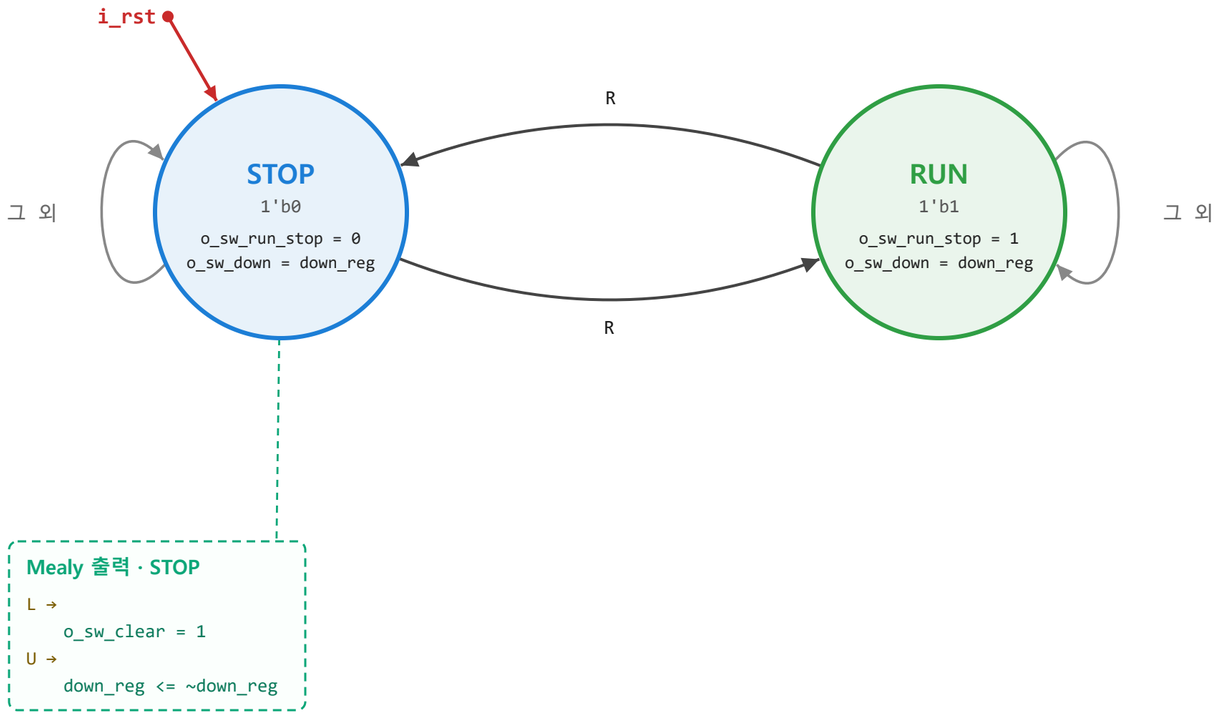
그림 = 코드 보증

- 모든 그림 · 표는 diagram/gen_fsm.py 의 SPEC 하나에서 그림. 만들 때마다 아래를 확인하고, 하나라도 다르면 PDF 를 만들지 않음

① enum 상태 이름 · 리셋 상태 · 입력 / 출력 포트를 RTL 파일에서 직접 읽어 SPEC 과 비교
② 다음 상태 표: 상태마다 모든 입력 조합에서 조건이 정확히 하나만 맞음 / ASM 차트와 표가 같은 결과
③ Verilator 로 RTL 을 무작위 입력으로 돌려 매 클럭 c_state · n_state · 출력이 ASM 차트와 같음: 스톱워치 20000 클럭 (조합 16 개 전부) · 타이머 20000 클럭 (조합 96 개 전부) · 시계 20000 클럭 (조합 128 개 전부)

스톱워치 컨트롤 유닛 — 상태도 (FSM)

U_STOPWATCH_CONTROL_UNIT (stopwatch_control_unit.sv) · 입력 i_sw_btn_R, i_sw_btn_L, i_sw_btn_U · 출력 o_sw_run_stop, o_sw_clear, o_sw_down



원 = 상태 (이름 · 코드 · Moore 출력) · 굵은 화살표 = 다음 클럭에 옮겨 가는 조건 · 가는 화살표 = 그대로 (유지) · 점선 = Mealy 출력

상태 · Moore 출력 (c_state 에 따라)

상태	코드	Moore 출력
STOP (리셋)	1'b0	o_sw_run_stop = 0 o_sw_down = down_reg
RUN	1'b1	o_sw_run_stop = 1 o_sw_down = down_reg

다음 상태 n_state (위 줄부터 = 우선순위)

c_state	조건	n_state
STOP	i_sw_btn_R	RUN
	그 외	STOP (유지)
RUN	i_sw_btn_R	STOP
	그 외	RUN (유지)

Mealy 출력 (조건이 1 인 그 클럭에 바로 · 표에 없는 출력 = 0)

c_state	조건	출력
STOP	i_sw_btn_L	o_sw_clear = 1
STOP	i_sw_btn_U	down_reg <= ~down_reg

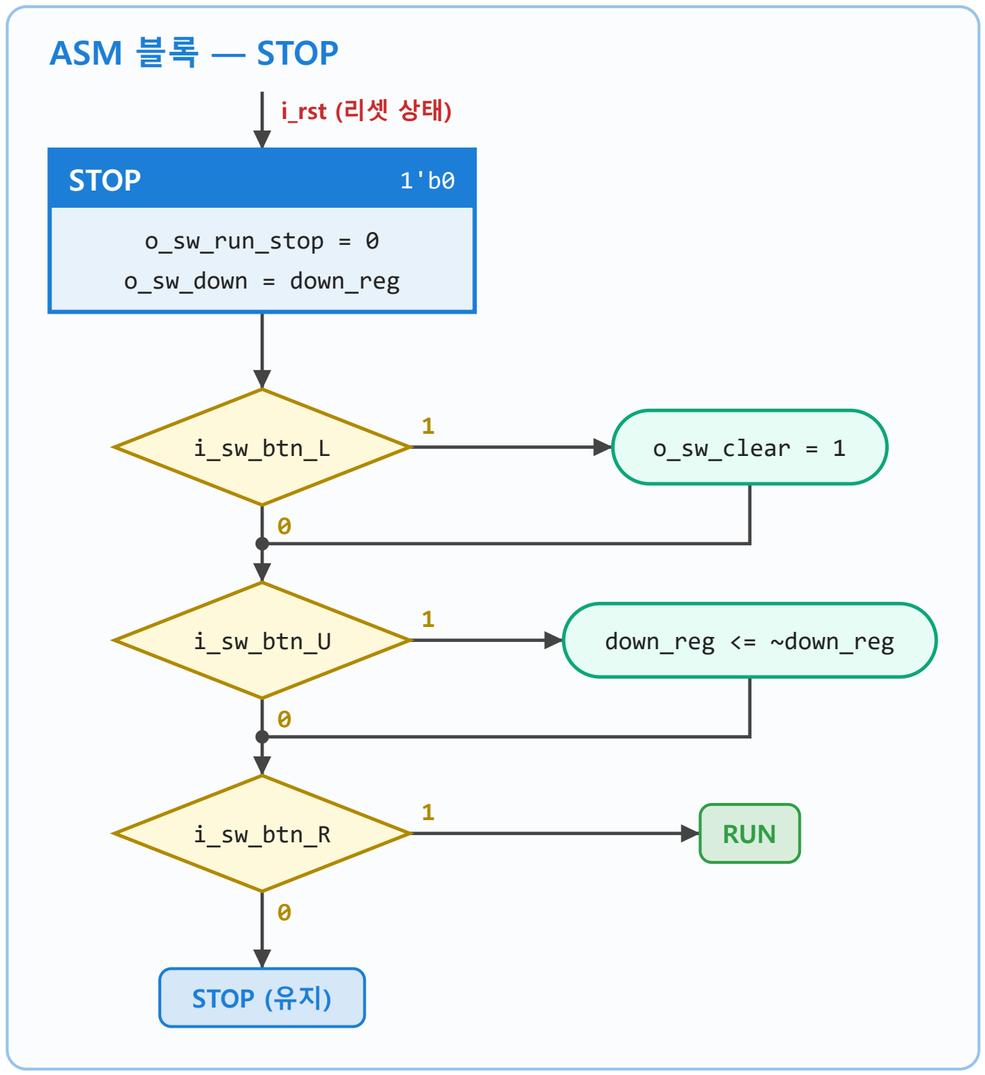
상태도 약어 · 메모

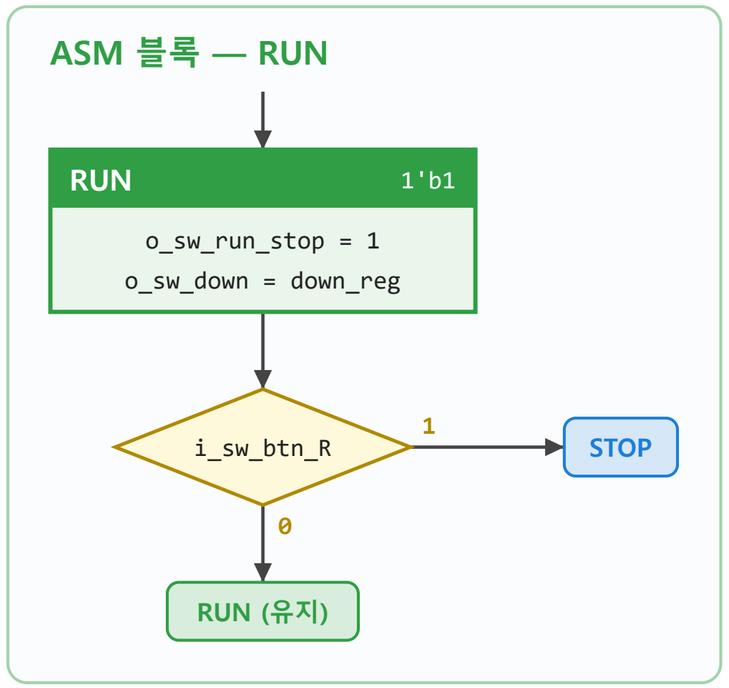
R = i_sw_btn_R L = i_sw_btn_L U = i_sw_btn_U

- o_sw_down 은 상태가 아니라 레지스터 down_reg (리셋 0) — STOP 에서 U 가 오면 다음 클럭 엣지에 뒤집힘
- RUN 중에는 L (지우기) · U (업/다운) 무시, R 만 봄
- D (랩) 은 컨트롤 유닛을 거치지 않고 stopwatch_datapath 로 바로 감

스톱워치 컨트롤 유닛 — ASM 차트

U_STOPWATCH_CONTROL_UNIT (stopwatch_control_unit.sv) · 상태 2 개 · ASM 블록 1 개 = 상태 1 개 (한 클럭)





상태 박스: 상태 이름 · 코드 + Moore 출력 (그 상태에 있는 동안 계속)

판단: 입력 조건 (1 / 0 갈래)

조건 출력 (Mealy): 그 클럭에 바로, 1 클럭 동안

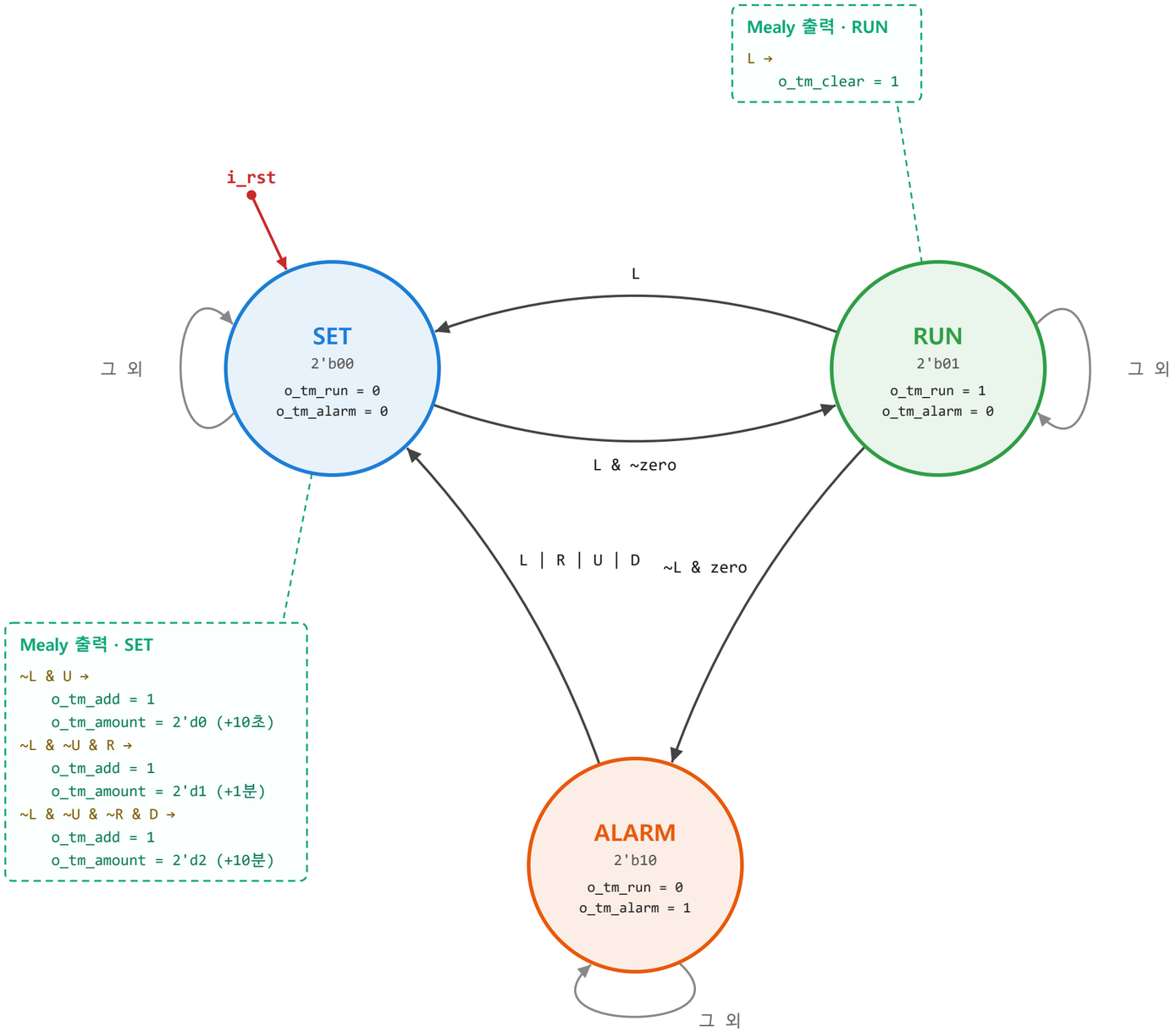
다음 상태 (다음 클럭 엣지에 c_state 가 됨)

표에 없는 출력 = 0 (기본값). 판단 순서 = RTL 의 if / else if 우선순위 (점으로 합류하는 판단은 서로 독립). <= 는 레지스터 (다음 클럭 엣지에 바뀜).

이 차트는 Verilator 로 RTL (stopwatch_control_unit.sv) 과 매 클럭 대조해서 같을 때만 만들어짐 (상태 × 입력 조합 전부).

타이머 컨트롤 유닛 — 상태도 (FSM)

U_TIMER_CONTROL_UNIT (timer_control_unit.sv) · 입력 i_tm_btn_R, i_tm_btn_L, i_tm_btn_U, i_tm_btn_D, i_tm_zero · 출력 o_tm_run, o_tm_alarm, o_tm_clear, o_tm_add, o_tm_amount



원 = 상태 (이름 · 코드 · Moore 출력) · 굵은 화살표 = 다음 클럭에 옮겨 가는 조건 · 가는 화살표 = 그대로 (유지) · 점선 = Mealy 출력

상태 · Moore 출력 (c_state 에 따라)

상태	코드	Moore 출력
SET (리셋)	2'b00	o_tm_run = 0 o_tm_alarm = 0
RUN	2'b01	o_tm_run = 1 o_tm_alarm = 0
ALARM	2'b10	o_tm_run = 0 o_tm_alarm = 1

다음 상태 n_state (위 줄부터 = 우선순위)

c_state	조건	n_state
SET	i_tm_btn_L & ~i_tm_zero	RUN
	그 외	SET (유지)
RUN	i_tm_btn_L	SET
	~i_tm_btn_L & i_tm_zero	ALARM
	그 외	RUN (유지)
ALARM	i_tm_btn_L | i_tm_btn_R | i_tm_btn_U | i_tm_btn_D	SET
	그 외	ALARM (유지)

Mealy 출력 (조건이 1 인 그 클럭에 바로 · 표에 없는 출력 = 0)

c_state	조건	출력
SET	~i_tm_btn_L & i_tm_btn_U	o_tm_add = 1 o_tm_amount = 2'd0 (+10초)
SET	~i_tm_btn_L & ~i_tm_btn_U & i_tm_btn_R	o_tm_add = 1 o_tm_amount = 2'd1 (+1분)
SET	~i_tm_btn_L & ~i_tm_btn_U & ~i_tm_btn_R & i_tm_btn_D	o_tm_add = 1 o_tm_amount = 2'd2 (+10분)
RUN	i_tm_btn_L	o_tm_clear = 1

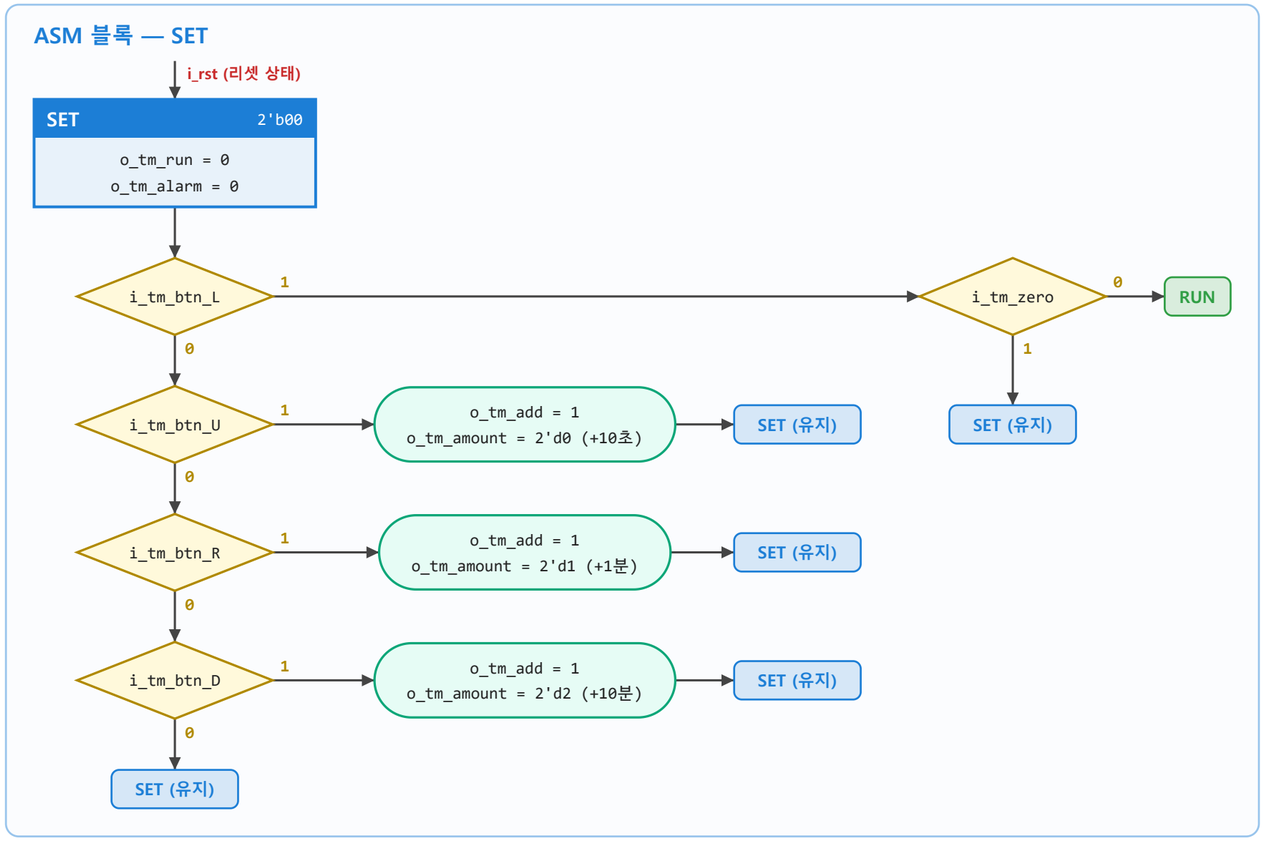
상태도 약어 · 메모

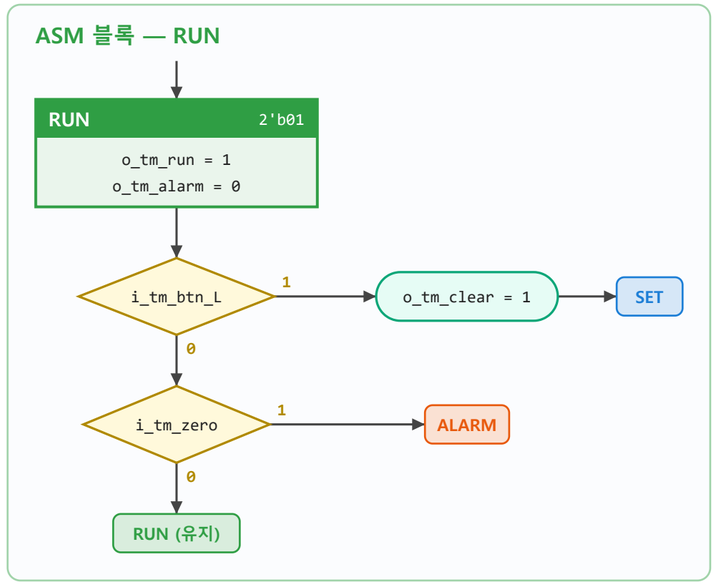
R = i_tm_btn_R L = i_tm_btn_L U = i_tm_btn_U D = i_tm_btn_D zero = i_tm_zero

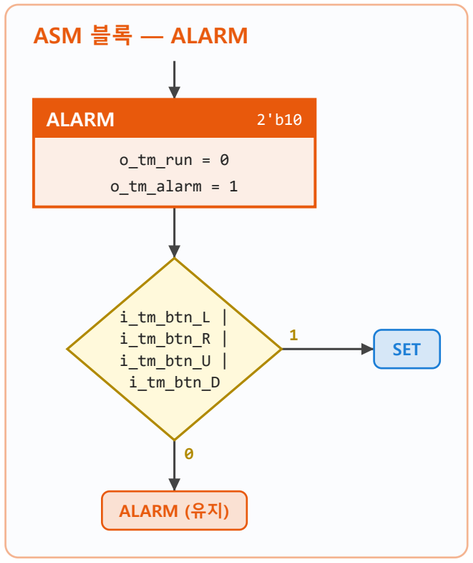
- SET 의 버튼 우선순위 L > U > R > D (같은 클럭에 둘 이상이면 앞의 것 하나만)
- SET 에서 L 이어도 i_tm_zero = 1 (남은 시간 0) 이면 시작 안 함
- i_tm_zero 는 timer_datapath 에서 옴 (남은 시간 = 0 이면 1)
- RUN 중 L = 취소: SET 으로 가면서 o_tm_clear 로 남은 시간 지우기

타이머 컨트롤 유닛 — ASM 차트

U_TIMER_CONTROL_UNIT (timer_control_unit.sv) · 상태 3 개 · ASM 블록 1 개 = 상태 1 개 (한 클릭)







상태 박스: 상태 이름 · 코드 + Moore 출력 (그 상태에 있는 동안 계속)

판단: 입력 조건 (1 / 0 갈래)

조건 출력 (Mealy): 그 클럭에 바로, 1 클럭 동안

다음 상태 (다음 클럭 엣지에 c_state 가 됨)

표에 없는 출력 = 0 (기본값). 판단 순서 = RTL 의 if / else if 우선순위 (점으로 합류하는 판단은 서로 독립). <= 는 레지스터 (다음 클럭 엣지에 바뀜).

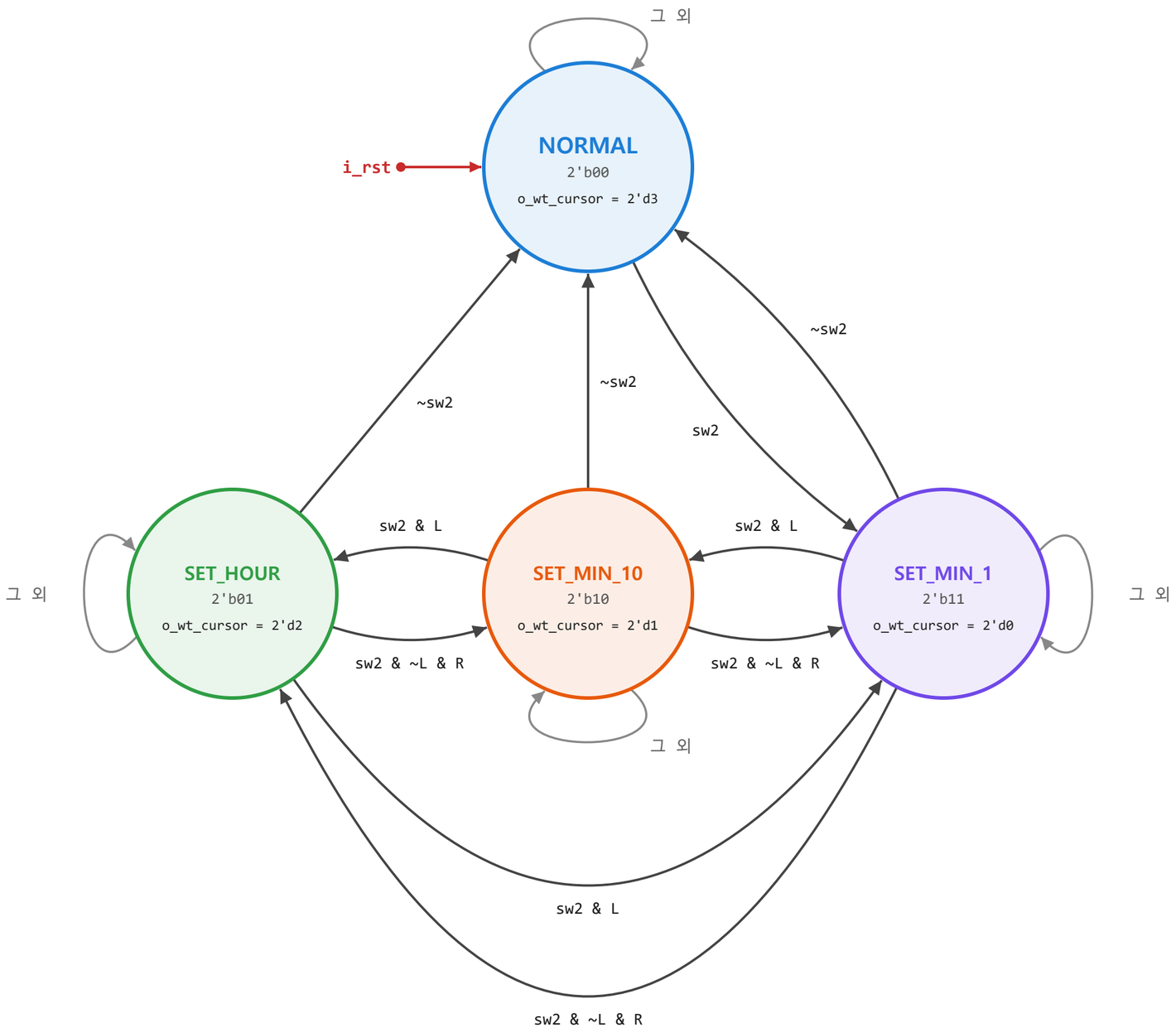
이 차트는 Verilator 로 RTL (timer_control_unit.sv) 과 매 클럭 대조해서 같을 때만 만들어짐 (상태 × 입력 조합 전부).

시계 컨트롤 유닛 — 상태도 (FSM)

U_WATCH_CONTROL_UNIT (watch_control_unit.sv) · 입력 i_wt_btn_R, i_wt_btn_L, i_wt_btn_U, i_wt_btn_D, i_set_sw2 · 출력 o_wt_cursor, o_wt_up, o_wt_down

Mealy 출력 · SET_HOUR · SET_MIN_10 · SET_MIN_1

```
sw2 & U & ~D →
    o_wt_up = 1
sw2 & D & ~U →
    o_wt_down = 1
```



원 = 상태 (이름 · 코드 · Moore 출력) · 굵은 화살표 = 다음 클럭에 옮겨 가는 조건 · 가는 화살표 = 그대로 (유지) · 점선 = Mealy 출력

상태 · Moore 출력 (c_state 에 따라)

상태	코드	Moore 출력
NORMAL (리셋)	2'b00	o_wt_cursor = 2'd3 (CUR_NONE)
SET_HOUR	2'b01	o_wt_cursor = 2'd2 (CUR_HOUR)
SET_MIN_10	2'b10	o_wt_cursor = 2'd1 (CUR_MIN_10)
SET_MIN_1	2'b11	o_wt_cursor = 2'd0 (CUR_MIN_1)

다음 상태 n_state (위 줄부터 = 우선순위)

c_state	조건	n_state
NORMAL	i_set_sw2	SET_MIN_1
	그 외	NORMAL (유지)
SET_HOUR	~i_set_sw2	NORMAL
	i_set_sw2 & i_wt_btn_L	SET_MIN_1
	i_set_sw2 & ~i_wt_btn_L & i_wt_btn_R	SET_MIN_10
	그 외	SET_HOUR (유지)
SET_MIN_10	~i_set_sw2	NORMAL
	i_set_sw2 & i_wt_btn_L	SET_HOUR
	i_set_sw2 & ~i_wt_btn_L & i_wt_btn_R	SET_MIN_1
	그 외	SET_MIN_10 (유지)
SET_MIN_1	~i_set_sw2	NORMAL
	i_set_sw2 & i_wt_btn_L	SET_MIN_10
	i_set_sw2 & ~i_wt_btn_L & i_wt_btn_R	SET_HOUR
	그 외	SET_MIN_1 (유지)

Mealy 출력 (조건이 1 인 그 클럭에 바로 · 표에 없는 출력 = 0)

c_state	조건	출력
SET_HOUR SET_MIN_10 SET_MIN_1	i_set_sw2 & i_wt_btn_U & ~i_wt_btn_D	o_wt_up = 1
SET_HOUR SET_MIN_10 SET_MIN_1	i_set_sw2 & i_wt_btn_D & ~i_wt_btn_U	o_wt_down = 1

상태도 약어 · 메모

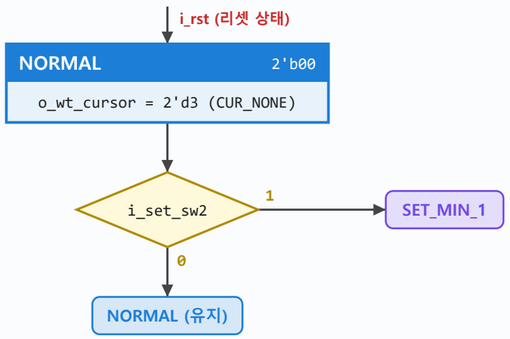
R = i_wt_btn_R L = i_wt_btn_L U = i_wt_btn_U D = i_wt_btn_D sw2 = i_set_sw2

- i_set_sw2 를 1 로 올리면 항상 SET_MIN_1 (FND 맨 오른쪽 자리) 부터, 0 으로 내리면 어느 자리에서든 NORMAL
- L 이 R 보다 우선, U 와 D 가 같은 클럭에 오면 무효 (o_wt_up · o_wt_down 둘 다 0)
- o_wt_up · o_wt_down 은 L · R 과 상관없이 나감 (같은 클럭에 L 과 U 가 오면 자리 이동 + 증가 둘 다)
- o_wt_cursor = FND 자리 번호 (AN0 = 맨 오른쪽): 0 분 일 · 1 분 십 · 2 시 일 · 3 없음

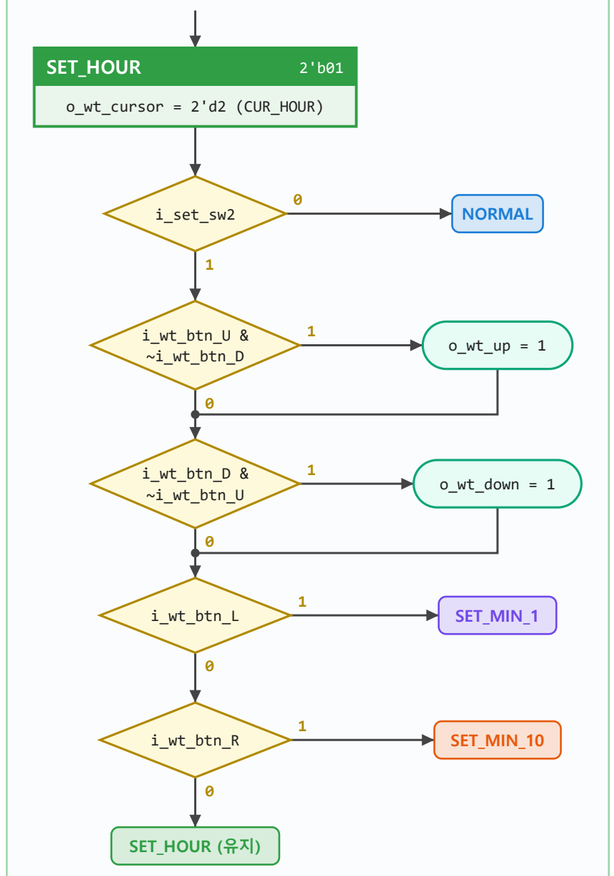
시계 컨트롤 유닛 — ASM 차트

U_WATCH_CONTROL_UNIT (watch_control_unit.sv) · 상태 4 개 · ASM 블록 1 개 = 상태 1 개 (한 클럭)

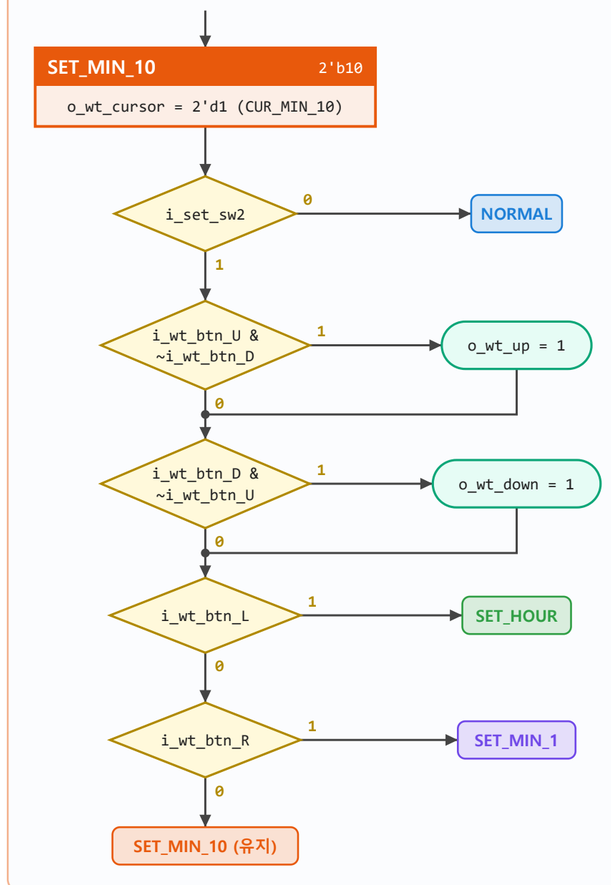
ASM 블록 — NORMAL



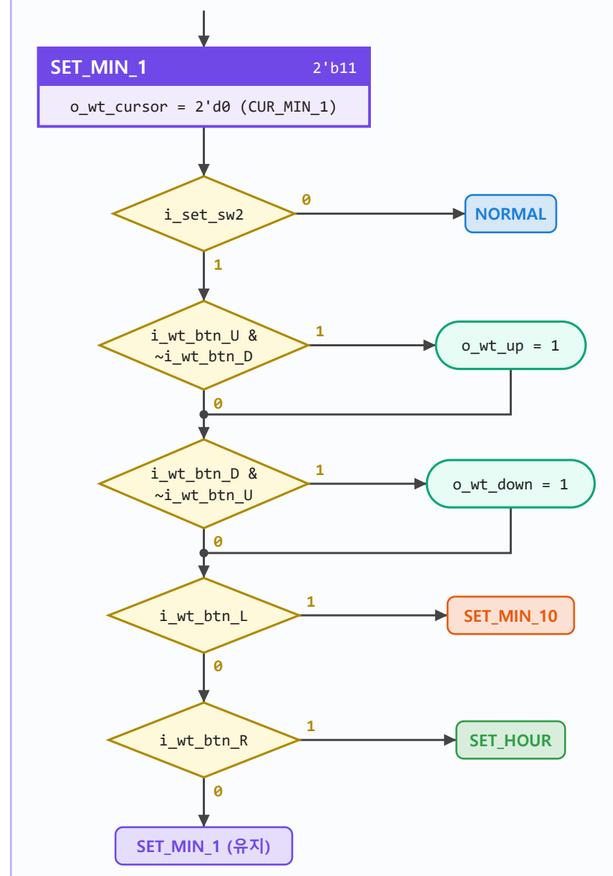
ASM 블록 — SET_HOUR



ASM 블록 — SET_MIN_10



ASM 블록 — SET_MIN_1



상태 박스: 상태 이름 · 코드 + Moore 출력 (그 상태에 있는 동안 계속)

판단: 입력 조건 (1 / 0 갈래)

조건 출력 (Mealy): 그 클럭에 바로, 1 클럭 동안

다음 상태 (다음 클럭 엣지에 c_state 가 됨)

표에 없는 출력 = 0 (기본값). 판단 순서 = RTL 의 if / else if 우선순위 (점으로 합류하는 판단은 서로 독립). <= 는 레지스터 (다음 클럭 엣지에 바뀜).

이 차트는 Verilator 로 RTL (watch_control_unit.sv) 과 매 클럭 대조해서 같을 때만 만들어짐 (상태 × 입력 조합 전부).